Extended MetaGPT Framework for AI-Generation of Legacy Systems

Introduction

Autonomous systems driven by Large Language Model (LLM) agents have broken ground in producing software systems that replicate human workflows and reasoning behaviour. However, many such frameworks tend to oversimplify the complex requirements involved in large-scale systems. For this industrial software project, we extended an open-source multi-agent framework called MetaGPT. Our aim was to address an existing gap in the functionality of the framework. MetaGPT is composed of a narrative-driven code generation pipeline that consumes user input prompts and generates deliverable source code. However, standard MetaGPT only supports basic, self-contained tasks that can be executed with a simple prompt, such as "Write a CLI blackjack game". The fundamental gap is that the framework is not properly equipped to develop deliverable source code for large-scale legacy systems, such as financial management systems.

Our proposed project details how to extend MetaGPT so that the framework scales up and can adjust to such legacy systems. To demonstrate and validate this framework, we partnered with Volopa as an industrial customer. Volopa is an expanding business payment and expense management platform which includes corporate cards and foreign currency exchange (FX) services. An existing functional gap in the platform concerns a corporate employee incurring an out-of-pocket (OOP) transaction via personal means such as hand-to-hand cash exchange or personal credit cards. Such payments made on behalf of the company should be tracked in the same manner as regular platform transactions.

We aim to extend our local installation of the MetaGPT framework and demonstrate an AI-based code generation pipeline for the Laravel-based source code for an OOP expense feature for the Volopa platform. The extensions proposed for this framework include customised MetaGPT roles, an integrated context-dimension layer and an evidence-gated verification protocol for the Laravel based system. These extensions were operationalised using a customised Python runner script, that initiates this pipeline and injects the necessary context into each agent. In our experiment, the pipeline successfully generated a structured Laravel-based codebase comprised of the Laravel OOP expense feature. This demonstrates that this pipeline can be utilised as a reusable process for scaling LLM-based code generation up to large-scale systems.

Contents

Introduction 1

Background 2

- MetaGPT Framework 2
- Volopa 3

Methodology 4

- Context Dimension Approach 4

Verification Protocol 5
Implementation 6
MetaGPT Runner Script Operationalisation 6
Context Layer Implementation 7
Context Extraction and Representation 7
Context Reader & Context Injection 7
MetaGPT Modifications 8
Custom Agents & Context Allocation 8
Additional Customized MetaGPT Actions and Schemas 10
Evidence-Gated Verification Loop 10
CHECK PLAN Emission 11
RAG Verification 11
Evidence-Gate Evaluation 12
Evidence Injection 13
Results 13
Conclusion 14
References 14
Annex 1 16
Annex 2 28

Background

MetaGPT Framework

MetaGPT [1] [2] is an open-source LLM-powered multi-agent framework designed to orchestrate multiple AI roles to collaboratively develop a single software engineering task. The various AI-powered roles, or agents that operate within this framework include a ProductManager, Architect, ProjectManager, Engineer and a QaEngineer. These agents are assigned to a MetaGPT Team, a modular environment wherein the agents cooperate. This framework essentially models a team of employees working at a software engineering company.

In the context of the framework, agents are referred to as roles, specifically the AI roles which cooperate to develop a requested software system. Basic tasks for agents can be established by defining the agent's roles, instructions and operational skills in user message prompts. Each role is wired to execute one or more MetaGPT Actions which are specifically suited to said role. These Actions are designed to perform specific tasks such as writing requirements documents, generating source code, or other tasks that require human cognition and reasoning abilities. A role can also be wired to watch for user message outputs published by other MetaGPT actions executed by previous roles. This enables multi-agent collaboration.

Within this MetaGPT framework [2], all agents share a global messaging pool. During the execution of the framework, an initial agent, a ProductManager reads an input natural language prompt message, executes WritePRD(), a defined method wired to said agent, which generates a corresponding output artifact, a product requirement document (PRD). This generated artifact is wrapped in a message context that is subsequently published in a structured format in the messaging pool where all roles store their output artifacts. Agents can also be wired to read the context messages published by other agents. Consequently, this MetaGPT execution flow models an assembly line of narrative artifact documents wherein each agent reads an input message from the shared message pool and generates a progressively larger artifact. The PRD document would be accessed then consumed by the Architect agent. This is due to the Architect being coded to observe the output context from the WritePRD() function. Once the WritePRD() function completes, this triggers the Architect agent. Subsequently, the Architect runs its wired action, WriteDesign() and generates a system design document which is then consumed by downstream agents, including the ProjectManager, Engineer and QaEngineer, which respectively generate modular software task lists, the specified source code and unit tests. An illustration of this collaborating framework is displayed in Figure 1.

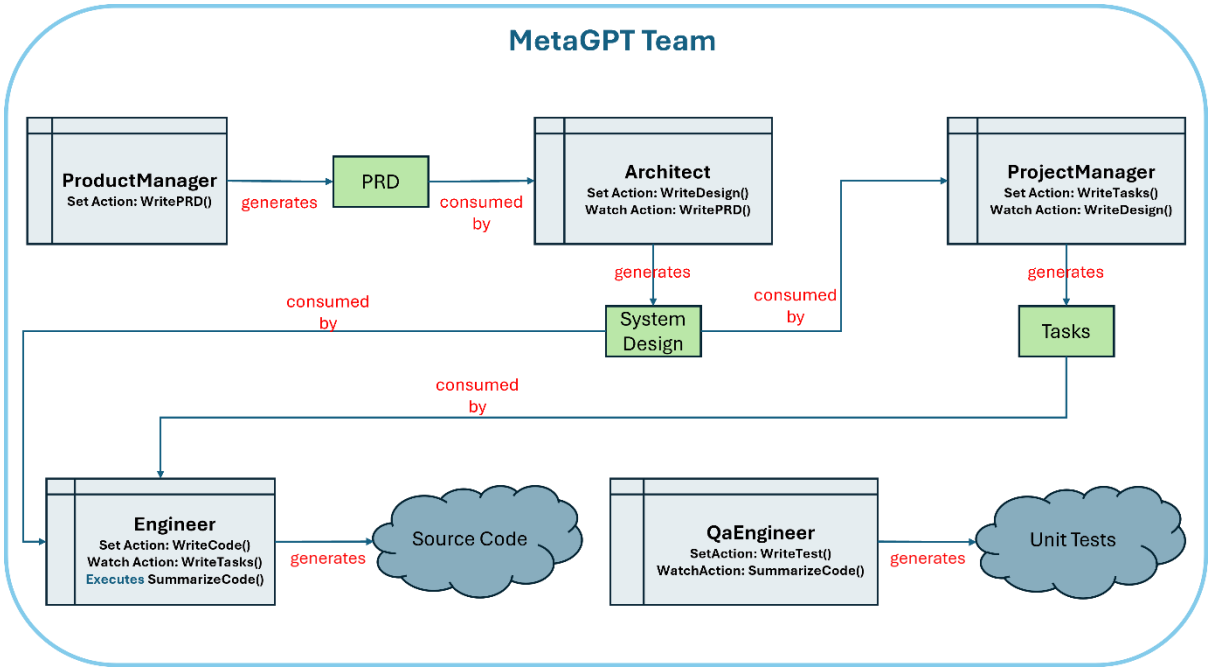

Figure 1 MetaGPT Environment & Pipeline

Volopa

Volopa is a financial technology, or fintech, company which provides a unified platform that provides global expenses, corporate card, foreign currency exchange services for corporate clients or businesses to assist in managing employee spending and international transactions. The Volopa platform works in a multi-tenant architecture, supporting multiple organisations with distinct user roles, permission hierarchies, and configurable financial workflows.

A key characteristic of the Volopa system is the presence of complex, interdependent constraints. These include role-based access control, transaction validation rules, multi-currency handling with FX conversion, and integration with existing platform services such as

authentication middleware and financial data infrastructure. As a result, new features must be developed in alignment with both platform-level conventions and feature-specific business logic.

The OOP expense feature highlights these characteristics. Unlike the standard Volopa card-based transactions, OOP expenses are handled through personal payment transactions. These out-of-office payments must be manually recorded, validated and integrated into existing workflows. To support this feature requires the coordination of multiple existing Volopa system components including expense data capture, user permission management and FX conversion logic.

Methodology

Context Dimension Approach

In the modern software engineering landscape, many software projects are defined by complex and heterogeneous industrial specifications. However, these context specifications are rarely delivered in an abstract single-layered structure. They can be divided into several conceptual dimensions: business objectives, rules and limits, functional and technical requirements, system constraints, database tables, structures and schemas, authorization constraints, user interaction flows and architectural decisions. In practise, these dimensions are often interwoven with output documents and artifacts. This results in difficulty while navigating and reasoning about individual aspects of the system.

Many software projects developed for the same organisational domain also tend to share context across project workspaces. Such platform-based context includes standard deployment targets, organisational policies and repository conventions. Furthermore, each individual project has their own unique components including problem statement, scope, domain glossary and entities. When a project source code is compiled, run and tested, all these differing requirements and constraints risk becoming inter-mixed which introduces ambiguity, varied interpretation of context and feature drift.

In a multi-agent generation platform, multiple components, or multiple LLMs are operationalised to drive pipelines. As a result of these multiple moving parts in a system's pipeline, context is at risk of being mis-interpreted independently across components. This leads to inconsistency, loss of implicit constraints, feature drift and divergence across components and their expected output artifacts.

To address this limitation, this study adopts a context-dimension layer methodology that is based on a canonical concept layer. This layer serves as a shared, structured representation of all context specifications across agents. Rather than allowing each agent to implicitly reconstruct context independently from narrative documents, this layer provides a single authoritative source of truth which all agents extract and interpret context from for all generation purposes. This prevents diverging interpretation of the same context across agents.

Within this canonical layer, input specifications are organized along two scopes of context:

- **Environment context:** captures shared, stable environment requirements that apply across all domain-specific projects regardless of the feature being developed
- **Project context:** captures unique, feature-specific specifications, including problem statements, domain entities, workflows, and system behaviour

This separation of context ensures that platform-level requirements and constraints remain distinct from project-level requirements reducing the risk of conflating global rules with project specific intent.

Within both these scopes, context can be decomposed further into a set of fixed semantic dimensions, each representing a single aspect of the system's truth and ensures each element is categorized according to its role in the system. These dimensions are defined as follows:

- **Intent**: "Why does this system exist?" and "What defines success for the system?"
- **Requirements:** Externally observable behaviour, what user roles and systems should do.
- **Constraints:** Hard functional and technical limits that restrict the project solution space.
- **Data Interfaces:** Structural contracts and schemas.
- **Flows:** Project-specific workflows that capture sequential behaviours.
- **Decisions:** Architectural commitments, trade-offs and open questions.

These defined dimensions prevent different categories of specifications from inter-mixing within a single artifact. This enables clearer reasoning, improves traceability and enables agents to reason over well-defined sub-sets of specifications relevant to the agent's role rather than a dense conflation of said specifications.

Overall, this approach represents a shift from narrative driven specification to shared structured context modelling. Narrative documentation such as requirement documents and design specifications is no longer treated as the primary source of truth, but as derived views from the underlying context model. This improves consistency across all generated artifacts, reduces feature churn and supports scalable multi-agent collaboration.

A detailed formalisation of this context-centric model, including its application and its mapping to conventional narrative-driven frameworks, is provided in Annex 1.

Verification Protocol

Within our context-dimension approach, constraints represent a set of non-negotiable rules or the hard functional and technical limits that restrict the system solution space. Ensuring that said constraints for output artifacts are obeyed is critical to maintaining the correctness and validity of the system, particularly when generating the source code.

This study introduces a verification protocol as a methodological mechanism that enforces code compliance before generation. The protocol mandates that compliance is made explicit and evidence-based rather than making assumptions that artifact integrity constraints are implicitly obeyed. This protocol declares that all system components referenced during artifact generation, referred to as symbols, must be treated as explicitly classified entities. These symbols include methods, functions, data structures, interfaces and other identifiable system components. Each symbol is classified according to its defined origin using the following taxonomy:

- **EXISTING:** symbols that are already present within the system repository
- **NEW:** symbols that are introduced as part of the proposed project context
- **UNKNOWN:** symbols whose status has not yet been resolved

This classification ensures that all symbols are explicitly identified and resolved before the system proceeds to code generation.

This verification protocol introduces a pre-generation verification step in which all symbols to be referenced during artifact generation are declared, classified and verified against an authoritative source of truth. For symbols expected to already exist, their presence must be confirmed before referencing them. For symbols that are intended to be introduced, it must be confirmed that no conflicting or duplicate code already exists. Any symbols whose status is uncertain must be resolved during the verification step before further steps are taken.

Generation of source code is only permitted once all symbols are grounded in verified information. This prevents system failure modes including referencing non-existent code, preventing duplications of existing functionality or introducing any inconsistencies in the codebase.

This verification protocol ensures that code generation only operates under evidence-enforced constraints rather than relying on implicit assumptions. This strengthens the reliability and integrity of the generated code, aligns this code with pre-existing systems and reduces the risk of structural errors during implementation.

A detailed operationalisation of this verification protocol, including the validation workflow is provided in Annex 2.

Implementation

Our research study extends the open-source MetaGPT framework [2] over the three core components: a defined Python runner script, the canonical context layer, and extended MetaGPT roles, schemas and actions. This innovation of this customized MetaGPT framework is to extend the standard code generation pipeline so that the framework scales up to large-scale legacy systems. For implementing this study, the open-source default MetaGPT multi-agent framework source code was downloaded, compiled and executed within our local workspace.

MetaGPT Runner Script Operationalisation

The standard MetaGPT framework is typically run for basic, self-contained tasks specified using a simple message. For an example, MetaGPT can generate a computerized blackjack game by typing a simple seed prompt message “write a cli blackjack game” in the terminal. This task is operationalized and fed to the MetaGPT pipeline as follows:

```
metagpt “write a cli blackjack game”
```

Once the MetaGPT environment initializes the required agents, including the ProductManager, Architect, and the Engineer, the pipeline proceeds to carry out said task based on this simple seed prompt. The output artifact, a single blackjack.py, is then stored in the relative workspace.

However, for this research project, the seed prompt instructions, the defined source and input specifications required for this requested multi-purpose financial legacy system are far too complex to be defined in a single sentence. The specifications include existing platform dependencies, infrastructure, domain requirements and precise database schemas and standard Volopa model schemas. The input prompt for the pipeline must reference these input source modules so that these required modules are not invented by the LLM. Therefore, we developed a custom Python runner script that instantiates the MetaGPT framework and enables us to inject the input specifications into the system prompt. This custom script is executed in the terminal as follows:

```
python .\industry\run_volopa_oop_expenses.py
```

In our approach, the MetaGPT source code is executed by this runner script that introduces the defined seed prompt in a string variable. This seed prompt declares that all input specifications derived from the customer contract requirements are contained within YAML source input files and are dynamically injected into each agent's system prompt. Once the script is executed, this seed prompt is broadcast to all MetaGPT roles in the MetaGPT environment simultaneously via the shared message pool, however only the ProductManager agent consumes this seed prompt. Each subsequent role in the MetaGPT workflow receives the message in their buffers, but do not act on it. Each of these agents, the Architect, the ProjectManager and the Engineer only consume the output artifact of each of their predecessors as illustrated in Figure 1.

Context Layer Implementation

To operationalise the context-dimension proposal described in the Methodology, a structured context layer was introduced as a foundational concept for this extended MetaGPT pipeline. This context layer externalises all input specifications into a unified machine-readable representation required by the system. Rather than treating all narrative generated output MetaGPT artifacts as a primary source of truth, this context layer redefines them as derived outputs from an underlying context model. This context layer acts as an intermediary between the input YAML sources and the source code generation pipeline. This ensures all specification context is accessible, structured and reusable across all system components.

Context Extraction and Representation

For this research project, the input context specifications were sourced from provided source files [3], [4], [5], and [6]. These documents contained complex, heterogeneous information about the proposed system, including business requirements, rules, system constraints, functional and architectural decisions, data schemas and detailed workflows.

All stated and unstructured requirement specifications were extracted from these files, then transformed into structured, atomic statements. Subsequently, these specifications were systematically categorized first along the two context scopes, environment and project, and then decomposed further into the six fixed defined dimensions, intent, requirements, constraints, data interfaces, flows and decisions which we defined earlier in our Methodology section. These structured context elements are materialised and stored in the following source YAML files:

- **Environment_context.yaml**[7] – contains the standardized domain-level requirements featured across all projects
- **Project_context.yaml**[8] – mutable project-specific context unique to the current implementation

Within both files, all elements were stored under the six conceptual dimensions forming a consistent and extensive context schema. This context representation supports maintainability, reusability and ensures that standardized environment level requirements remained distinct while allowing mutable project-level statements to evolve across different use cases.

Context Reader & Context Injection

To enable the MetaGPT pipeline to utilise this structured representation, a context access or context-reader mechanism was introduced in the workspace to mediate integration between agents and the context layer. Rather than expose the total context entirely across all agents of the

pipeline, this context-reader utility supports controlled query-based access and retrieval of specific subsets of source context based on agent requirements.

This approach transforms input context from a static input into actively managed resource within the pipeline. Agents do not consume raw, unstructured context data, they consume role-oriented views derived from the canonical context layer. These views are dynamically injected into each agent's standard MetaGPT system prompt and are thus integrated into the agent's reasoning process. This process ensures that relevant context is accessed and is consistently interpreted by each agent.

Therefore, this context-reader utility functions as a bridge between the YAML input context and the framework's roles. By appending only role-specific views derived from the input context, this approach reduces costly and unnecessary implementation, ambiguous context and improves the quality of the generated outputs.

MetaGPT Modifications

The MetaGPT framework, outlined in our Background section, supports agents that operate as user-based roles and execute assigned actions as illustrated in Figure 1. In our local extension, these roles integrate with the context-reader utility so that the appropriate context views are accessed and consumed by the relevant LLM-based agents.

To implement this framework, the following MetaGPT roles, ProductManager, Architect, ProjectManager, Engineer and QaEngineer, were customised and extended in our workspace in order to acclimate the Laravel-based environment to support Volopa. Our custom roles extend their respective standard MetaGPT role and functionality via Python subclassing. To promote maintainability and usability throughout our execution of the MetaGPT framework, we also extended and modified various MetaGPT actions and artifact output schemas wired for these agents.

Custom Agents & Context Allocation

The context-dimension layer described in the Methodology section defines the structured and canonical representation of all system specifications. To ensure this context approach is used within our MetaGPT framework, the input context source must be dynamically distributed and consumed by agents.

Our custom MetaGPT roles were extended to consume structured context in a controlled and role-specific manner. Rather than relying solely on narrative artifacts as the primary source of truth, each agent is also provided with a filtered role-focused view of the canonical context layer. Each view of the input context is aligned with the role-specific responsibility within the pipeline.

In our workspace, each of our custom roles instantiates the shared context-reader utility and operates over a subset of context accessor functionality that fulfils all role-focused views of the YAML context while maintaining consistency with the default pipeline. This represents a shift in default MetaGPT behaviour where LLM agents sequentially interpret output artifacts, to a model where context is explicitly partitioned and dynamically allocated per agent.

The allocation of YAML context is followed via the dual-scope structured categorisation introduced in Methodology: **environment-context** where stable platform standards and conventions are shared across projects, and **project-context** capturing feature-specific requirements and behaviour. Within these scopes, context is further decomposed into the fixed

semantic dimensions: intent, requirements, constraints, data interfaces, flows and decisions. Each role is provided only the context required to complete its task.

For an example, our extended ProductManager.py role operates with specific functionality to query and access the role-specific dimensions of YAML context for generating a product requirements document (PRD). As the first role for a code generation pipeline, a ProductManager operates on intent, requirements and high-level constraints which reflect business level objectives and system boundaries. The following agent in the pipeline, the Architect consumes a much broader view, operating on lower-level constraints, interfaces, dynamic system workflows and architectural and functional decisions. These context dimensions are essential for the construction of the system design specification that aligns with platform-based standards and project-specific features. Downstream agents including the ProjectManager, Engineer and the QaEngineer would operate with a stronger emphasis on constraints, interfaces and decisions, ensuring the generated source code adheres to defined system rules and structures. This role-based context allocation is formalised through a mapping between the defined agents and context dimensions is outlined in Table 1 below.

Table 1 Role-based Context Allocation

Roles	**Context-Scope**	**Dimensions Accessed**	**Purpose**
ProductManager	Environment + Project	Intent, Requirements, Constraints	Define system goals and expected behaviour
Architect	Environment + Project	Interfaces, Flows, Constraints, Decisions	Design system structure and workflows
ProjectManager	Environment + Project	Constraints, Interfaces, Decisions	Plan implementation tasks and dependencies
Engineer	Environment + Project	Constraints, Interfaces, Decisions	Generate compliant source code
QaEngineer	Environment + Project	Requirements, Constraints, Flows	Validate behaviour and system correctness

In our runner script defined above, these extended MetaGPT roles views were registered with our MetaGPT environment instead of the default roles. Each role instantiates the context-reader utility defined above and operates over the pre-defined context accessor functionality to append the role-specific context to the LLM's reasoning ability for artifact generation.

This structured allocation of context enhances the scalability of the MetaGPT framework by enabling it to operate over complex heterogeneous specifications typical of large-scale legacy systems. This reduces agent ambiguity, reduces context drift and supports more reliable coordination between roles.

Additional Customized MetaGPT Actions and Schemas

To further support this AI generation of the Laravel-based source code, the standard MetaGPT actions and schemas for developing the system design and the source code artifacts were extended at a conceptual level. These modifications were focused on improving control, maintaining consistency and alignment between the system design artifact and the generated source code.

At a high-level, default MetaGPT code-generation behaviour was designed to enforce stricter adherence to upstream artifacts in the pipeline before code generation. Instead of encouraging exhaustive or fully completed implementations, the code-generation process was constrained to operate within clearly defined design boundaries. This ensured that generated outputs remained consistent with explicitly defined artifact structures, thus avoiding the introduction of inferred or unreferenced functionality. Wherever input specifications extracted from input context source were unclear or ambiguous in any output artifacts, the respective LLM-agent was prompted to preserve placeholder gaps in the generated source code rather than implicitly resolving them, thus reducing the risk of hallucinated code.

Furthermore, the MetaGPT pipeline was also extended so that the intermediate system design artifacts were equipped to better attend to the constraints-focused context rather than receiving and interpreting them passively in an agent's system prompt. Instead of repeatedly injecting constraints context across multiple agents, thus leaving the constraints context open to re-interpretation by multiple LLM-agents, the constraint material is consolidated and formalised during the system design phase thus being properly consumed by downstream agents which now consume a single consistent and explicit representation of the critical system functional and technical rules and limits. By embedding the constraints directly within the shared system design output as illustrated in Figure 1, the modified framework reduced risk of reinterpretation.

These modifications to the default MetaGPT pipeline serve to:

- reduce hallucinations and over-generation in output source code
- encourage alignment between system design and output source code artifacts
- provide consistency in how constraints are maintained across agents
- support incomplete code due to unclear or evolving specifications

Evidence-Gated Verification Loop

To operationalise our Verification Protocol described in Section 2, the system introduces an evidence-gated verification loop. This evidence gate layer for the verification step implements core principles including pre-generation code compliance, explicit symbol classification and evidence-backed constraint enforcement, ensuring that all referenced Laravel elements are verified prior to code generation. Rather than allowing code generation to proceed based on a prompt, this gated layer ensures that code generation is grounded in verified information rather than implicit assumptions.

This verification protocol is implemented in our extended custom Engineer role which extends the default MetaGPT Engineer agent. Our custom Engineer agent is prompted to not reference any symbol until its existence has been verified against a live repository or it verified to be introduced by the proposed system alteration. Therefore, our customised Engineer agent not only

instantiates the context-reader utility as with our other custom MetaGPT agents but also implements the verification step in the form of an evidence-gated verification loop. This verification protocol is injected into the system prompt for the agent during initialisation after being accessed by our context access layer.

CHECK PLAN Emission

To implement the pre-generation code validation requirement, the agent is prompted by the verification protocol to generate a structured CHECK PLAN. During the execution workflow, this CHECK PLAN element only exists in the runtime or the execution layer of the code-writing agent, so it is not a generated design output that is saved in the output source code directory. It is a mandatory pre-generation artifact as well as a disposable execution scaffolding of the Laravel-based symbols the system intends to reference, query, and subsequently generate once the verification step is concluded.

This CHECK PLAN lists the Laravel-based symbols the agent intends to query against the live repository, the symbol's 'origin' tag, which can be either 'EXISTING', 'NEW', or 'UNKNOWN' and their respective verification functions to be run against the RAG repo. A sample CHECK PLAN is illustrated in Figure 2.

```
CHECK PLAN
- PaymentFileUpload  -> NEW     -> collision check
- UploadController   -> UNKNOWN -> RAG lookup
- ProcessCsvJob      -> UNKNOWN -> RAG lookup
- CsvValidator       -> UNKNOWN -> RAG lookup
```

Figure 2 CHECK PLAN Example

These listed symbols in the CHECK PLAN are tagged as either 'NEW' or 'EXISTING'. These origin values are derived from the YAML context source files. This initial CHECK PLAN is also comprised of symbols that were assigned the origin type 'UNKNOWN' because they were generated by the LLM-agent during the system design artifact and thus were not listed and not tagged in the input specifications. These 'UNKNOWN' symbols thus need to be resolved against the RAG repository and reclassified as either 'EXISTING' or 'NEW'. The verification protocol also declares and assigns a required verification function for each symbol depending on their assigned origin and are outlined as follows:

- 'EXISTING': "MUST cite RAG evidence using function 'rag_symbol_lookup'"
- 'NEW': "MUST verify the symbol does not already exist via 'rag_collision_check'"
- 'UNKNOWN': "MUST perform 'rag_symbol_lookup' to resolve the symbol as either 'EXISTING' (hit(s) found) or 'NEW' (0 hits + collision check passes)"

RAG Verification

Now that the CHECK PLAN has been declared, it is intercepted by the agent's runtime or execution layer. Once intercepted, the runtime deterministically parses the CHECK PLAN, then extracts the listed Laravel symbols, each symbol's origin tag and its verification function as structured items in a linear fashion.

Subsequently, the agent composes these extracted values into structured queries sent to the live OpenSearch index 'volopa_platform_symbols' which functions as a RAG repository and the query returns JSON hits that are designated and formatted as cited evidence for verification result for each queried symbol. The returned hits are formatted as cited evidence for each of the four

queried symbols in a CHECK PLAN from Figure 2 above are formatted as cited evidence and listed in Figure 3.

```
RAG RESULT 1: UploadController
  Found at: app/Http/Controllers/UploadController.php
  Public methods: upload(), validate()

RAG RESULT 2: ProcessCsvJob
  Found at: app/Jobs/ProcessCsvJob.php
  Constructor args: ($uploadId)

RAG RESULT 3: CsvValidator
  Not found in repository.

RAG RESULT 4: PaymentFileUpload
  No matching class or table found.
```

Figure 3 Sample RAG Results

As illustrated in Figure 3, the formatted RAG evidence returns hits for the two symbols 'app/Http/Controllers/UploadController.php' and 'app/Jobs/ProcessCsvJob.php', thus confirming these symbols already exist in the repository and the agent is prevented from creating duplicated code for these requested symbols and the 'UNKNOWN' origin for 'ProcessCsvJob' is reclassified as 'EXISTING'. Also, the symbol 'PaymentFileUpload' is confirmed to be newly introduced by the proposed system change and has passed the collision check function as mandated by the verification function.

Using this evidence, the agent cross-references each symbol's initial origin tags against the returned RAG hits and then re-classifies any symbol in the CHECK PLAN that is initially incorrect with the cited evidence. The agent also reclassifies any symbol with the origin 'UNKNOWN' as either 'NEW' or 'EXISTING' and cites these returned RAG hits.

Evidence-Gate Evaluation

The agent must confirm that all symbols in the CHECK PLAN have passed this verification step before proceeding to code generation. Once all evidence hits are returned, the runtime flows into an evidence-gated verification loop. This is an evidence-based evaluation mechanism that only allows the agent to proceed to code generation once all queried symbols pass the evaluation criteria.

As an example, if any symbols were declared 'EXISTING' but no hits were found in the RAG index, the evaluation fails because these supposedly EXISTING symbols have been unverified. Also, the evaluation will fail if any symbols were re-classified as NEW, but the respective queries returned cited hits meaning these symbols already exist in the RAG index, resulting in duplicate functionality. The total deterministic pass/fail rules for this evaluation step are defined in Table 2:

Table 2 Verification Evaluation Rules

Designated Origin	RAG Hits	Resulting Classification
EXISTING	Found	PASS (verified)
NEW	Not Found	PASS (safe to create)
UNKNOWN	Found	PASS (reclassify as EXISTING)
UNKNOWN	Not Found	PASS (reclassify as NEW)

EXISTING	Not Found	FAIL (symbol is unverified)
NEW	Found	FAIL (symbol collision)

If any of these FAIL modes are present amongst the referenced symbols to generate in the evaluation step, the agent is re-prompted to re-generate the CHECK PLAN again and correct these specific failures from the previous attempt.

Once all symbols are verified against the RAG and there is determined to be no duplicate code collisions and no remaining unverified symbols then the runtime exits this verification loop and proceeds to code generation. Our extended Engineer agent is prompted to have only three tries to pass this verification. If all three attempts fail, then the custom MetaGPT workflow is halted completely, and no source code is generated.

Evidence Injection

Now that all symbols in the CHECK PLAN have been verified, the formatted evidence hits for all symbol queries must be injected into the agent's action prompt context. Due to the RAG system being an external component, the cited evidence from the index is not visible to the LLM-agent. The LLM for our extended Engineer role must be able to see these evidence-based hits to proceed to code generation.

The customized Engineer then appends these structured evidence results as observations to the system design-focused content for the customized LaravelWriteCode action's context. This ensures that the evidence hits for each queried symbol are re-injected into the agent's prompt. Now that the evidence-based evaluation is appended to the agent's prompt, the runtime proceeds to code generation for all elements declared in the system design artifact.

Results

In our research study, the custom runner script was successfully executed, running our extended MetaGPT pipeline, the Laravel-based source code generation workflow. Upon completion of the script, the complete workflow spanning CHECK PLAN emission, the verification of referenced Laravel symbols against an external RAG, sequential multi-agent artifact and source code generation was completed without system failure.

During the code generation phase, the evidence-gated verification loop operated as intended. The workflow passed through the verification loop twice. The Laravel symbol verification step failed on the first attempt due to a single unverified symbol, an Eloquent model called 'Feature' which was declared EXISTING but returned no hits from the RAG index. Due to this, the system blocked the code generation and re-prompted the Engineer agent. Upon the second attempt the verification gate passed and proceeded to code generation. This behaviour validates our verification protocol which enforces the constraint that all referenced generated files must be grounded in verified information before code generation proceeds. It also enforces the correction of any misclassified files before source code is produced, preventing errors from being propagated across the generated codebase. Upon completion of the runner script, the extended MetaGPT pipeline was shown to produce a structured codebase composed of sixty-five Laravel symbols (controllers, models, database migrations, service classes, API resource classes, a single background job class) across the application. The generated workspace also included a set of artifact documents including the PRD, the system design and the task breakdown, and the

generated source code was stored in the designated workspace output directory and is publicly available in this repository [9].

The successful execution of our custom pipeline operationalised by our runner script, combined with the observed behaviour of the verification protocol demonstrates that the extended MetaGPT framework can support the generation of structured, multi-component implementations aligned with complex, real-world system specifications.

Conclusion

This research study demonstrated that MetaGPT framework can be extended to support the generation of structured, multi-component software systems with complex, real-world standards and conventions. Thus, by extending MetaGPT with customised agents, actions and schemas and by integrating with context-driven orchestration, context-based dimension reasoning and verification-gated generation, a large-scale system that requires complex, heterogeneous platform and feature-specific specifications can be generated.

These outputs and resulting source code indicate that context-dimension modelling and partitioning and distribution of appropriate context across agents can improve consistency and reduce feature churning and hallucinations across multi-agent code generation pipelines. Furthermore, the addition of a verification protocol ensures that all generated code is grounded in verifiable knowledge mitigating common failures such as invalid assumptions and hallucinations.

However, it should be noted that there are several limitations to this study. As stated above in the Methodology section, several context specifications provided by the client are ambiguous and unresolved, and thus the source code generated includes placeholder code which need to be updated once these unclear specifications are clarified. Thus, this MetaGPT framework source code is essentially a prototype which we intend to build upon. Other limitations are that the external OpenSearch index used for the verification step was a locally deployed RAG system and the input YAML source context files that compose the context-dimension layer required manual transformation from the source files provided by the client. These factors introduce constraints on scalability that would need to be addressed in future work.

Despite these limitations, this case study has proven that the standard MetaGPT pipeline can be systematically extended to operate over complex, multi-dimensional specifications. By shifting from user-provided prompt driven-generation to a context-driven orchestration model, this custom MetaGPT framework provides an approach for applying LLM-based multi-agent frameworks for the generation of large-scale legacy software systems.

References

[1] "MetaGPT | MetaGPT." Accessed: Apr. 13, 2026. [Online]. Available: https://docs.deepwisdom.ai/main/en/

[2] *FoundationAgents/MetaGPT*. (Apr. 13, 2026). Python. FoundationAgents. Accessed: Apr. 13, 2026. [Online]. Available: https://github.com/FoundationAgents/MetaGPT

[3] "metaGPT_Volopa_OOP/industry/requirements/SD-OOP_Pocket_expense.pdf at main · stephen-devops/metaGPT_Volopa_OOP," GitHub. Accessed: Apr. 13, 2026. [Online]. Available: https://github.com/stephen-

devops/metaGPT_Volopa_OOP/blob/main/industry/requirements/SD-OOP_Pocket_expense.pdf
[4] "metaGPT_Volopa_OOP/industry/requirements/SD-OOP_Expense_single_data_capturing.pdf at main · stephen-devops/metaGPT_Volopa_OOP," GitHub. Accessed: Apr. 13, 2026. [Online]. Available: https://github.com/stephen-devops/metaGPT_Volopa_OOP/blob/main/industry/requirements/SD-OOP_Expense_single_data_capturing.pdf
[5] "metaGPT_Volopa_OOP/industry/requirements/SD-OOP_User_Management_System.pdf at main · stephen-devops/metaGPT_Volopa_OOP," GitHub. Accessed: Apr. 13, 2026. [Online]. Available: https://github.com/stephen-devops/metaGPT_Volopa_OOP/blob/main/industry/requirements/SD-OOP_User_Management_System.pdf
[6] "metaGPT_Volopa_OOP/industry/dos_and_donts.pdf at main · stephen-devops/metaGPT_Volopa_OOP," GitHub. Accessed: Apr. 13, 2026. [Online]. Available: https://github.com/stephen-devops/metaGPT_Volopa_OOP/blob/main/industry/dos_and_donts.pdf
[7] "metaGPT_Volopa_OOP/industry/requirements/environment_context.yaml at main · stephen-devops/metaGPT_Volopa_OOP," GitHub. Accessed: Apr. 13, 2026. [Online]. Available: https://github.com/stephen-devops/metaGPT_Volopa_OOP/blob/main/industry/requirements/environment_context.yaml
[8] "metaGPT_Volopa_OOP/industry/requirements/project_context.yaml at main · stephen-devops/metaGPT_Volopa_OOP," GitHub. Accessed: Apr. 13, 2026. [Online]. Available: https://github.com/stephen-devops/metaGPT_Volopa_OOP/blob/main/industry/requirements/project_context.yaml
[9] stephen-devops, *stephen-devops/metaGPT_Volopa_OOP*. (Apr. 10, 2026). Python. Accessed: Apr. 14, 2026. [Online]. Available: https://github.com/stephen-devops/metaGPT_Volopa_OOP

Annex 1

Adjusting MetaGPT for use with Legacy Systems

Caveat: This is based on an analysis of the Mass Payment pptx and extrapolates the inferred design properties to the general design environment, which of course may not be the actual case.

1. Introduction

Recent advances in large language models have enabled the emergence of agentic software engineering frameworks capable of synthesizing complex artifacts from compact prompts. MetaGPT exemplifies this trend by orchestrating multiple agents, typically a Product Manager and an Architect, to generate progressively richer outputs such as product requirement documents and system design specifications. These outputs are expressed in natural language augmented by diagrams and structured conventions embedded in prompts. The attraction of this approach lies in its low upfront modeling cost and its apparent ability to bootstrap meaningful designs rapidly from limited user input.

However, as soon as the input problem moves beyond a narrow demonstration and approaches the complexity of real industrial specifications, the limitations of narrative-driven artifact generation become apparent. Industrial requirements rarely arrive in a cleanly layered form; instead, they intermix business objectives, user interaction flows, validation rules, regulatory or organizational constraints, interface expectations, and partial architectural assumptions in a single corpus. The Mass Payments specification is illustrative: it includes CSV schema rules, approval concurrency semantics, asynchronous processing expectations, HTTP response behavior, user navigation flows, and references to controllers and background jobs. When such material is processed purely as text by agent pipelines, implicit structure must be rediscovered repeatedly, which introduces ambiguity, inconsistency, and drift across generated artifacts.

This paper compares two models for addressing this problem. The first is the baseline MetaGPT workflow, which relies on narrative documents as the primary medium for knowledge transfer between agents. The second is a context-centric model that introduces an explicit canonical representation of system knowledge structured along six fixed dimensions and shared across all agents. The objective is not to replace MetaGPT's roles or procedural logic, but to formalize the information those roles already manipulate in order to scale reliably to complex systems.

1.1 The Baseline MetaGPT Approach

MetaGPT is built around a role-based, sequential workflow. A small number of agents – most notably a Product Manager and an Architect – collaborate to expand a minimal user input (often a single-line requirement) into progressively richer artifacts. The Product Manager generates a Product Requirements Document (PRD). The Architect consumes the PRD and produces a

system design document. Downstream agents may then generate code, tests, or deployment artifacts.

What matters structurally is not the surface appearance of the documents, but the implicit semantic dimensions embedded in the prompts.

From the PRD prompt you shared, the Product Manager is asked to produce:

- Project metadata and restated requirements.
- Product goals.
- User stories.
- Competitive analysis and quadrant chart.
- Technical requirements analysis.
- A prioritized requirements pool.
- A UI design draft.
- Open questions.

From the Architect prompts and examples, the Architect is asked to produce:

- An implementation approach.
- A file list (physical structure).
- Data structures and interfaces (UML-like class diagrams and APIs).
- Program call flow (sequence diagrams and runtime behavior).
- Anything unclear (open issues).

These outputs are not arbitrary; they correspond to real design dimensions: intent, user behavior, competitive positioning, technical constraints, interface contracts, data models, runtime flows, architectural structure, and unresolved risks.

However, MetaGPT treats these dimensions as **implicitly embedded in narrative documents**. The PRD and system design are the only durable artifacts. There is no explicit canonical representation of constraints, flows, interfaces, or decisions outside those documents. Each agent reconstructs understanding from free text. As complexity increases, this creates several systemic issues:

- **Implicit context drift**: constraints embedded in prose or diagrams are reinterpreted by each agent.
- **Feature creep**: new sections or ad-hoc artifacts are introduced as projects become more complex.
- **Ambiguity propagation**: unclear assumptions propagate downstream instead of being stabilized early.
- **Scaling limits**: the “one-line requirement” abstraction works for toy systems but breaks for enterprise-grade specifications like Mass Payments.

MetaGPT’s strength is that it is lightweight and fast to bootstrap. Its weakness is that it relies on the LLM’s internal coherence to maintain consistency across artifacts rather than on explicit structure.

1.2 The Context-Centric Six-Dimension Approach

The alternative approach you’ve been developing introduces a **canonical context layer** that sits alongside the agents. Instead of allowing each agent to reconstruct context implicitly from

prose documents, the system maintains a structured, shared representation of the system's "truth".

This model defines six stable dimensions:

1. **Intent**
2. **Requirements**
3. **Constraints**
4. **Flows**
5. **Interfaces**
6. **Decisions**

The key design choice is that these six dimensions are fixed. They do not change per project, per domain, or per organization. Only the content within them changes. This avoids schema churn and prevents the framework from accreting new ad-hoc artifacts over time.

A second key idea is the split between:

- **Environment context** – stable, cross-project rules, standards, and platform constraints.
- **Project context** – feature-specific intent, rules, flows, and interfaces.

In this model, PRDs and system designs are no longer the primary source of truth. They become narrative views generated from the canonical context, rather than the place where facts are invented and stored.

2. Mapping the Two Approaches

The table below links the explicit design dimensions embedded in MetaGPT's prompts to the six canonical context dimensions.

Table 1 Mapping of design dimensions

MetaGPT Dimension	**Canonical Dimension**
Project metadata (language, templates, repo conventions)	Constraints / Intent
Product goals	Intent
User stories / actors	Requirements
Competitive analysis	Intent
Requirements analysis	Constraints
Requirements pool / prioritization	Requirements

UI design draft	Requirements
Implementation approach	Decisions
File list / physical layout	Decisions
Data structures and interfaces (UML, APIs)	Interfaces
Program call flow (sequence, lifecycle)	Flows
MetaGPT Dimension	**Canonical Dimension**
Open questions / unclear	Constraints or Decisions

This mapping is lossless. Nothing that MetaGPT currently generates disappears. Instead, it becomes explicitly classified.

The Mass Payments PPT illustrates why this matters. That document mixes:

- Business objectives (enable batch payments).
- User flows (upload → validate → approve → process).
- Validation rules and limits (row caps, conditional fields).
- Concurrency semantics (first approver claims).
- API contracts and status behaviors.
- Architectural hints (controllers, jobs, queues).

In the baseline MetaGPT flow, all of this lands in a PRD or system design narrative and must be reinterpreted. In the context model, each element is routed deterministically into one of the six dimensions.

3 The Six Dimensions in Detail

What follows is a comprehensive description of each dimension, including which artifacts from the Product Manager prompt, Architect prompt, and Mass Payments materials are absorbed by each.

3.1 Intent

Intent captures **why the system exists and what success means**. It is not a feature list. It is purpose, motivation, and positioning.

From the PRD prompt, Intent absorbs:

- Product goals.
- Competitive analysis and quadrant positioning.
- The problem framing implicit in the restated requirements.

From the Mass Payments material, Intent includes:

- The objective of enabling scalable batch payments.
- The need to support operational efficiency and approval control.
- The business rationale for automation versus manual entry.

Intent does not encode implementation detail or constraints. It is the anchor against which design tradeoffs are evaluated.

3.2 Requirements

Requirements describe **externally observable behavior**: what users and external systems can do. From the PRD prompt, this includes:

- User stories.
- Requirements pool (prioritized features).
- UI design draft (screens, interactions, workflows visible to users).

From the Mass Payments PPT, this includes:

- Uploading CSV files.
- Validating rows and showing errors.
- Viewing uploaded files and their statuses.
- Approval actions and user navigation flows.
- Downloading templates.

Requirements intentionally remain technology-agnostic and user-centric.

3.3 Constraints

Constraints define **hard rules that restrict solution space**. These include technical limits, policy mandates, compliance rules, and unresolved uncertainties.

From the PRD prompt:

- Technical requirements analysis (performance expectations, limits).
- Open questions (treated as unresolved constraints until clarified).
- Project metadata that mandates certain languages or templates.

From the Mass Payments materials:

- Maximum CSV row counts.
- Conditional validation rules by currency and recipient type.
- Concurrency semantics (first approver claims).
- Status code behavior (404 vs 200).
- Asynchronous processing mandates.
- Any organizational “do/don’t” rules.

Constraints are non-negotiable unless explicitly changed.

3.4 Flows

Flows capture **dynamic behavior and sequencing**: how the system evolves over time.

From the Architect prompt:

- Program call flow (sequence diagrams).
- Object lifecycle behavior.

From the Mass Payments PPT:

- Upload → validate → queue → approve → process.
- Approval claiming race conditions.
- Status polling.
- Notification timing.

Flows are not environment artifacts; they are always project-specific because they encode business process semantics.

3.5 Interfaces

Interfaces define **structural contracts and schemas**.

From the Architect prompt:

- UML class diagrams.
- Method signatures.
- API interfaces.

From the Mass Payments PPT:

- API endpoints and response codes.
- CSV column schemas and formats.
- Entity relationships (uploads, errors, instructions).

UML class models naturally live here as the authoritative structural view, while invariants or ownership semantics are normalized into Constraints or Decisions.

3.6 Decisions

Decisions capture **architectural commitments and tradeoffs**.

From the Architect prompt:

- Implementation approach.
- File structure.
- Technology choices.

From the Mass Payments case:

- Using background jobs for CSV processing.
- Service boundaries.
- Persistence strategy.
- Handling of concurrency mechanisms.
- Whether validation is synchronous or asynchronous.

Decisions include rationale and consequences, enabling traceability.

See also Appendix A.

4. Design Context

Consists of Environment and Project context

Environment Context

Environment context contains facts that apply across many projects:

- Coding standards and repository rules.
- Platform templates and frameworks.
- API conventions.
- Error envelope formats.
- Pagination rules.
- Observability standards.
- Organizational “do/don’t” policies.

These primarily populate:

- Constraints.
- Interfaces (platform-level contracts).
- Decisions (platform architecture).

They are stable and versioned slowly.

Project Context

Project context contains everything specific to a feature or system:

- Intent specific to the business problem.
- User requirements.
- Business rules and limits.
- Workflow semantics.
- Feature-specific APIs and schemas.
- Architectural decisions unique to the project.

This is where Mass Payments lives. Both contexts share the same six dimensions and are merged logically at runtime.

5. How Context is Represented (Conceptually)

The context is stored in the following sources

- **Environment context file** – this is presumably created once and amended as needed but not very often.
- Certain environment constraints may reference **external verification sources**, such as a repository RAG index, rather than embedding the underlying data directly in the context file. In such cases, the environment context specifies the policy and the verification mechanism, while the RAG provides the authoritative evidence at runtime. The RAG interface is probably exposed as a query service (?) that supports structured symbol and metadata lookup (e.g., existence, signatures, locations), and is invoked by agents or their runtime as part of constraint verification before generation proceeds.
- **Project context file** – this should be generated.

Each source contains the six dimensions as structured sections. They do not encode executable logic or code; they encode declarative facts: goals, rules, flows, schemas, and decisions.

Importantly:

- The schema is stable.
- Only content changes per project.
- Artifacts such as UML diagrams, sequence flows, and API schemas are embedded as structured text blocks or references.

This allows both humans and machines to reason over the same source of truth.

6. Injecting Context into LLM Agents

Rather than changing agent roles or adding new ones, the minimal integration approach is to **inject the merged context directly into the LLM prompts** used by the existing Product Manager, Architect, and Engineer agents.

Conceptually:

1. The system loads the environment and project context.
2. The two are merged into a unified canonical context.
3. This context is prepended to each agent's prompt as authoritative material.

4. The agent is instructed:

   - Not to redefine facts already in context. o To surface conflicts or uncertainties explicitly.
   - To treat constraints as hard rules.
   - To align outputs with flows and interfaces already defined.

The PRD agent then produces a narrative document that references this context rather than inventing it. The Architect produces designs that elaborate on decisions and structure but remain consistent with the canonical flows and interfaces. Downstream agents inherit the same shared understanding.

Rather than changing agent roles or adding new ones, the minimal integration approach is to inject the merged context directly into the LLM prompts used by the existing Product Manager, Architect, and Engineer agents.

Conceptually:

1. The system loads the environment and project context.
2. The two are merged into a unified canonical context.
3. This context is prepended to each agent's prompt as authoritative material.
4. The agent is instructed:

   - Not to redefine facts already in context. o To surface conflicts or uncertainties explicitly. o To treat constraints as hard rules. o To align outputs with flows and interfaces already defined.
   - Where constraints require external verification, to invoke the declared verification interface (e.g., repository RAG) and incorporate evidence before producing outputs.

No new roles are required. No major SOP rewrite is required. The system gains stability through explicit shared state rather than through prompt complexity.

7. Comparative Summary

The baseline MetaGPT approach relies on **implicit coherence** inside narrative documents. It is fast, flexible, and lightweight, but fragile under complexity.

The context-centric approach introduces **explicit canonical structure** without replacing existing workflows. It absorbs all existing MetaGPT design dimensions, including UML class models, flows, and architectural reasoning, while providing a stable foundation for scaling to real-world systems like Mass Payments.

The shift is architectural rather than algorithmic: it treats context as a first-class artifact rather than an emergent property of text generation.

Appendix A

Table 2 Comparison of Narrative-Driven MetaGPT Design vs Context-Centric Six-Dimension Mode

Dimension	**Narrative-Driven MetaGPT Workflow**	**Context-Centric Six-Dimension Model**
Primary representation of knowledge	Free-text documents (PRD, system design) containing mixed concerns. Semantics are implicit and reconstructed by each agent from narrative text and diagrams.	Canonical structured context explicitly organized into six fixed dimensions: Intent, Requirements, Constraints, Flows, Interfaces, Decisions. Narrative documents are views over this context.
Semantic stability	Implicit. Meaning is inferred from prompts and prose; interpretation may drift across agents and iterations.	Explicit. Each fact has a defined semantic category and stable schema; interpretation is consistent across agents.
Treatment of requirements	User stories, requirements pools, and UI drafts embedded in PRD prose; prioritization and scope are narrative constructs.	Requirements are first-class structured artifacts representing externally observable behavior and scope.

Dimension	**Narrative-Driven MetaGPT Workflow**	**Context-Centric Six-Dimension Model**
Treatment of intent and business rationale	Product goals and competitive analysis appear in PRD narrative but are not structurally linked to downstream design decisions.	Intent is explicitly modeled and traceable to requirements and decisions.
Treatment of constraints	Technical requirements and implicit rules appear in prose or diagrams; enforcement depends on agent interpretation.	Constraints are explicit canonical artifacts that restrict solution space and are injected into every agent prompt.
Representation of flows	Sequence diagrams generated by Architect exist as standalone artifacts without formal linkage to requirements or constraints.	Flows are canonical representations of lifecycle and sequencing, referenced consistently by all agents.
Representation of interfaces and data models	UML class diagrams and APIs embedded in system design documents; semantics may be	Interfaces are canonical structural contracts; UML diagrams live directly within the Interfaces dimension.

	duplicated or diverge from other sections.	
Architectural decisions	Implementation approach and file structure are embedded in system design narrative; rationale is informal and weakly traceable.	Decisions are explicit, versioned artifacts capturing commitments, tradeoffs, and rationale.
Handling of mixed specifications	Mixed concerns are interpreted heuristically by agents; ambiguity increases with document complexity.	Mixed specifications are mechanically decomposed and routed into canonical dimensions.
Environment-level rules	Repeated implicitly across projects or embedded in prompts and templates.	Captured once in environment context and reused consistently across projects.
Project-specific variability	Embedded directly in PRDs and designs without structural separation from platform rules.	Encoded in project context with clean separation from environment context.
Traceability	Implicit and narrative; relationships between intent, requirements, constraints, and decisions are not machine-explicit.	Explicit and machine-traceable across dimensions.
Dimension	**Narrative-Driven MetaGPT Workflow**	**Context-Centric Six-Dimension Model**
Scalability to complex systems	Degrades as specification density increases; prone to drift and duplication.	Scales predictably because semantic structure remains stable regardless of complexity.
Framework evolution	Tends toward schema creep and ad-hoc artifact creation as complexity grows.	Schema remains fixed; only content changes.
Integration effort	Minimal upfront modeling; relies on LLM coherence.	Small upfront modeling cost but reduced downstream instability and rework.

Role impact	Roles reconstruct context independently from narrative documents.	Roles consume the same canonical context without changing responsibilities.
Change propagation	Changes must be manually synchronized across documents.	Changes propagate automatically through shared context.
Automation potential	Limited machine reasoning over narrative artifacts.	Enables validation, consistency checking, and automated reasoning over structured context

Annex 2

CHECK PLAN, Constraints, and Verification Protocol

Operational design for repository-aware enforcement in MetaGPT agents

This document was developed through an iterative conversational process.

Original queries are shown as ▶ *red annotations* throughout.

Introduction and Motivation

▶ *Clarify — "Inject the policy (do/don't) as a constraint."*

LLM-based agents in MetaGPT generate design and code by following instructions in their system prompt. When an agent must obey platform policies — such as "do not reference undefined symbols" — the policy must be injected as a binding constraint, not as a suggestion or style guideline.

To *inject a policy as a constraint* means to pass a do/don't rule to the LLM as a nonnegotiable instruction that governs how it must work. Mechanically:

- The policy appears in a dedicated **CONSTRAINTS** section of the system prompt.
- It is marked as **mandatory** and **must be checked before output**.
- It is paired with a protocol rule that says how compliance is demonstrated.

This works because LLMs are good at translating natural-language rules into action plans, deciding whether a rule needs a lookup or reasoning, and emitting structured evidence when required. What they are bad at is silently enforcing rules without accountability. The solution is to make the rule explicit, binding, and checkable, then require proof of compliance before output.

Injecting a policy as a constraint means telling the LLM: this rule is mandatory, you must check it, and you must show evidence before you produce output.

Foundational Concepts: Constraints and Protocol

▶ *Clarify "Inject the policy as a constraint" vs "Require the LLM to produce a CHECK PLAN before doing the work."*

The enforcement mechanism has two distinct, complementary parts that represent a separation of concerns inside the prompt.

Constraints — What Must Be True (Declarative)

Constraints answer: *What rule is non-negotiable?* They define the obligation without specifying steps or output structure, and apply globally for the whole task. Think of them as **law**.

CONSTRAINT

You must not reference any class or method unless its existence has been verified in the repository or it is explicitly introduced in this change.

The CHECK PLAN Protocol — How Compliance Is Demonstrated (Procedural)

The protocol answers: *How will you prove you obeyed the rule before doing the work?* It forces explicit reasoning, externalises "I checked this" into a visible artifact, and gives the runtime something to gate on. Think of it as **due process**.

PROTOCOL

Before generating code, you must produce a CHECK PLAN that lists:

- which constraints require verification
- what checks you will perform
- what evidence will be produced

Why Both Are Needed

If you only inject the constraint: the LLM knows the rule, but may silently assume it complied. There is no handle to enforce or verify compliance.

If you only require a CHECK PLAN without a constraint: the LLM may check irrelevant things or skip important policies, because nothing tells it what is mandatory.

Both are required: the rule (constraint) and the proof mechanism (CHECK PLAN). Together they produce the following flow:

1. LLM reads the constraint (what must hold).
2. LLM produces a CHECK PLAN (how it will ensure it holds).
3. Runtime executes any requested lookups/tools.
4. LLM generates code with evidence-backed confidence.

Stable vs Variable Parts

▶ *So we can extend the constraints section as needed but the protocol section stays the same?*

Yes. This is the core design principle.

CONSTRAINTS are **variable**. They are extended, removed, or changed per environment or project. They encode *what must be true* and derive from environment context and project context.

PROTOCOL is **stable**. It does not change per project. It encodes *how the LLM must behave* and is part of the agent's standard operating procedure.

The prompt always has the same procedural skeleton; only the **content of**

CONSTRAINTS evolves. New rules are data, not code. This gives extensibility without agent churn.

Symbol Classification: EXISTING, NEW, UNKNOWN

▶ *Some constraints may be project-local and some environment-related. When we merge both in the prompt, how do we distinguish new symbols from existing ones?*

When environment and project context are merged in the prompt, the agent must be able to distinguish between symbols that already exist and symbols it is expected to create. This is solved by forcing a classification step and binding it to verification rules.

The Origin Taxonomy

Every referenced symbol is classified into exactly one of three origins:

- **EXISTING:** Already in the repository (or verified to exist). Must cite evidence (path + signature).
- **NEW:** Introduced by this change. Must declare creation location and collisioncheck.
- **UNKNOWN:** Not yet verified. Must be checked via RAG to resolve to EXISTING or NEW.

This applies to classes, methods, functions, files, routes, database tables, and queue names.

Two Scopes of Constraints

Environment constraints (always-on policies). Examples: "Don't reference nonexistent repo symbols"; naming conventions; architectural mandates. These typically imply verification against external truth (repo/RAG).

Project constraints (feature rules). Examples: CSV columns; approval semantics; row limits; endpoint status rules. These typically imply internal consistency (flows, interfaces, requirements).

Even if physically merged into one prompt block, constraints should be labelled by scope.

How Scopes Interact with Origin

Environment-level constraints usually force checks for EXISTING. For example: "Don't reference a class unless it exists" requires a repo/RAG check whenever the origin is EXISTING or UNKNOWN.

Project-level constraints may introduce NEW symbols intentionally. For example, the project context may declare a new entity PaymentFileUpload. That symbol is NEW by project intent, so it should not trigger a "must already exist" check. Instead it triggers a collision check: ensure it doesn't already exist under another name. For updates to existing projects, the agent has two sources of truth:

- **Repo/RAG:** ground truth of what exists (environment + prior project code).
- **Project context:** what this change plans to add/change (declared NEW).

If project context lists an entity as NEW, treat it as NEW (still collision-check). If the agent invents a symbol not in project context, it is UNKNOWN → must check.

The LLM Performs Classification Dynamically

▶ *“Marks X as EXISTING or NEW.” I thought we need to tag symbols — but the LLM does so itself?*

There are two different “tagging” steps, and only one is done by the LLM. The LLM classifies symbols conceptually, as part of its reasoning and protocol compliance. This classification happens inside the model’s reasoning and outputs — not in the environment file and not in code.

The environment provides:

1. **The rule:** Every referenced symbol must be classified as EXISTING, NEW, or

UNKNOWN.

2. **The enforcement:** UNKNOWN → must query RAG. EXISTING → must cite evidence. NEW → must declare creation and collision-check.

3. **The verification interface:** Repo RAG is available for symbol lookup.

The LLM applies these rules at runtime. This keeps policy in the environment context, evidence in RAG/tool outputs, and judgement in the LLM — avoiding hard-coded enforcement logic, brittle symbol registries, and per-project schema creep.

Symbol classification (EXISTING, NEW, UNKNOWN) is performed dynamically by the agent according to the injected verification protocol and updated as verification evidence becomes available.

The Verification Architecture

RAG as a Declared Verification Source

▶ *Suppose our environment wants to use a RAG-encoded repo to enforce constraints. How would this extend or modify the environment representation?*

The repo contents are not embedded in the environment context file. Instead, the environment representation is extended so that RAG becomes a first-class verification mechanism. The key distinction is between:

- **Policy** (what must be true) — stays in the environment context.
- **Evidence source** (how you check it) — declared as a capability.

The environment file becomes a **control plane** for verification, not a data dump. It encodes:

1. What must be verified (policies/constraints).
2. How verification is performed (verification sources + query contract).
3. What evidence is required (protocol).

Structural Additions to the Environment Representation

Three stable constructs are added at the environment level:

Verification Source Declaration

Describes the RAG index as a resource:

- Coverage: what repos/branches/languages.
- Indexing granularity: symbols, signatures, files, call graph.
- Freshness: when last updated or required max staleness.
- Query contract: supported query types (symbol-exists, method-on-class, filepath, interface signature).

Constraint Annotation

Constraints requiring repository verification are tagged, not hard-coded into agent logic:

- Constraint: *"No undefined symbols."*
- Verification required: repo_symbol_lookup
- Failure mode: *"Block generation."*

Verification Protocol

A single, reusable procedure that any agent can follow:

- Produce CHECK PLAN.
- Execute checks using declared source(s).
- Attach evidence references.
- Only then emit design/code.

No new roles are required. Existing roles are extended so they know they must verify and where to verify.

The environment context remains the canonical specification of platform policies; the repository RAG index is introduced as a canonical verification resource. The environment representation is extended not by embedding repository contents but by declaring (i) verification sources, (ii) constraint-to-verification bindings, and (iii) a uniform verification protocol.

Runtime Verification: On-the-Fly, Not Pre-Injection

▶ *Is verification done before prompt injection or on the fly?*

▶ *Why is evidence in the prompt? Surely the constraints just say "check the RAG."*

▶ *So we should inject returned results as a prompt to the agent?*

Verification happens **during the agent's execution**, not before prompt injection. The clean sequencing is:

Phase A: Prompt injection (static)

- Environment context: policies + "use RAG to verify EXISTING."
- Project context: requirements + declared NEW symbols.

Phase B: On-the-fly verification (dynamic)

1. Agent produces a CHECK PLAN and identifies symbols needing verification.
2. Agent requests a verification query (e.g., “Check whether class X exists.”).
3. Runtime executes the query against the RAG index (outside the LLM).
4. Results are injected back into the agent’s prompt/conversation state.
5. Agent updates its reasoning, classifies symbols, and proceeds or requests the next check.
6. Repeat until all checks are satisfied.

This is a standard tool–observe–continue loop.

Evidence must be visible to the LLM because if the agent needs to generate design/code that depends on verification results, it must *see* those results. Otherwise it is forced to guess or hallucinate. RAG evidence is appended as observations mid-run, not embedded into the initial context injection (except for the rule “you must verify using RAG”).

Separate Evidence Injection per Query

▶ *Given that there may be many queries, we just inject each query result separately?*

Yes. Each verification query result is injected separately, as it becomes available, into the agent’s working prompt.

- **Keeps reasoning grounded.** Each result becomes visible to the model immediately, so subsequent decisions are conditioned on verified facts.
- **Preserves causal traceability.** You can audit which query produced which evidence and which decision depended on it.
- **Avoids over-fetching.** The agent only queries what it actually needs.
- **Supports early failure.** If a blocking constraint fails early, execution stops immediately.

Evidence Is Transient, Not Canonical

Evidence **does not modify the canonical context**. Context = policies + declared project facts. RAG results = transient evidence used to satisfy constraints. They live in different layers.

In practice: keep only relevant recent evidence in the active prompt window; optionally summarise older evidence once decisions are locked; evidence does not need to persist across roles unless it affects downstream constraints.

When a constraint requires external verification, the agent issues a query to the declared RAG interface, and the returned results are injected back into the agent’s prompt as observational evidence before generation proceeds.

Prompt Architecture

▶ *Would it be cleaner to separate them into sub-parts if they are merged into a single prompt?*

Yes — even if environment and project context are injected together, they should be presented as explicitly separated sections so that the LLM preserves scope boundaries.

Three-Part Prompt Structure

Environment Context (Stable, Always-On)

- Policies/standards (do/don't rules).
- Repo constraints (must verify against RAG).
- Platform conventions/templates.
- Verification capability declaration (RAG endpoint, query types).
- Universal protocol (CHECK PLAN, evidence requirement).

Project Context (Variable, Feature-Specific)

- Intent.
- Requirements.
- Constraints (project-local).
- Flows.
- Interfaces (including UML).
- Decisions/open questions.
- Declared NEW symbols (what this change is allowed/expected to introduce).

Evidence/Observations (Dynamic, Runtime)

- RAG query results (symbol exists, paths, signatures).
- File tree snapshots.
- Any tool outputs.

This third part is appended on the fly during agent execution.

Precedence Rules

- **Environment constraints override everything.**
- Project constraints override agent invention.
- Evidence resolves UNKNOWN → EXISTING/NEW decisions.
- If a symbol is in Project.NEW, it is allowed to be NEW (still collision-check). If not, it is UNKNOWN → must verify via RAG.

Why Separation Matters

- **Unambiguous precedence.** Each scope's authority is clear.

- **New vs existing is tractable.** The agent can follow a simple rule against the Project.NEW list.
- **Reduces accidental drift.** Headings and separation reduce “policy leaking” into “design facts” and vice versa.

Although environment and project context are injected together to provide a unified view, they are presented as explicitly separated sections (Environment, Project, and runtime Evidence) so that precedence, verification, and “new vs existing” classification rules remain deterministic.

The CHECK PLAN Artifact

▶ *What does a CHECK PLAN look like?*

▶ *Where does the agent output the CHECK PLAN?*

Purpose and Properties

A CHECK PLAN is a short, explicit list of symbols or artifacts that an agent intends to reference, their proposed origin (EXISTING, NEW, UNKNOWN), and the verification actions required before generation proceeds.

It should be small, mechanical, easy to validate, and tightly scoped to verification only. It is disposable execution scaffolding, not a design artifact.

Structure

Each entry has three mandatory parts:

1. Item to be referenced.
2. Proposed origin (EXISTING / NEW / UNKNOWN).
3. Required verification.

Optionally: which constraint triggered the check.

Text Format

CHECK PLAN

1. PaymentFileUpload

  Origin: NEW

Verification: Collision check in repo

2. UploadController Origin: EXISTING

Verification: Confirm class exists via repo RAG

3. CsvValidator

Origin: UNKNOWN

Verification: RAG lookup for symbol existence

4. process_csv_job Origin: UNKNOWN

Verification: RAG lookup for job naming/existence

Ultra-Compact Format

```
CHECK PLAN
    - PaymentFileUpload  -> NEW     -> collision check
    - UploadController   -> EXISTING -> verify in repo
    - CsvValidator       -> UNKNOWN  -> RAG lookup
    - process_csv_job    -> UNKNOWN  -> RAG lookup
```

JSON Format (Machine-Enforced Mode)


```
{
  "CHECK_PLAN": [
    {
      "id": "CP-001",
      "kind": "class",
      "name": "UploadController",
      "origin": "UNKNOWN",
      "verify": ["rag_symbol_lookup", "rag_signature_lookup"],
      "constraint": "ENV-CON: must not reference undefined symbols"
    },
    {
      "id": "CP-002",
      "kind": "class",
```

```
    "name": "ProcessCsvJob",
    "origin": "UNKNOWN",
    "verify": ["rag_symbol_lookup", "rag_signature_lookup"],
    "constraint": "ENV-CON: must not reference undefined symbols"
  },
  {
    "id": "CP-003",
    "kind": "class",
    "name": "PaymentFileUpload",
    "origin": "NEW",
    "verify": ["rag_collision_check"],
    "constraint": "ENV-CON: must not redefine existing symbols"
  }
 ]
}
```


Corresponding tool calls:


```
[
 { "command_name": "RepoRAG.lookup_symbol",
   "args": { "symbol": "UploadController" } },
 { "command_name": "RepoRAG.lookup_symbol",
   "args": { "symbol": "ProcessCsvJob" } },
 { "command_name": "RepoRAG.collision_check",
   "args": { "symbol": "PaymentFileUpload" } } ]
```


Where the Agent Emits the CHECK PLAN

The CHECK PLAN is emitted as the **first part of the agent's normal action output**, before any design or code generation, in the same channel where it would normally start writing. The runtime intercepts it, executes the required verifications, injects results, and resumes the agent.

What a CHECK PLAN Is Not

- Not a design explanation or requirements analysis.
- Not a reasoning chain or justification essay.
- Not stored in project context or written to disk.

- Not persisted across runs.
- Not visible to downstream agents unless explicitly propagated.

It is **runtime control scaffolding only**.

MetaGPT Integration

▶ *How do I actually implement this?*

▶ *What does the Engineer output today and how will this change it?*

▶ *Remind me what the third part of the changes is?*

The entire enforcement mechanism requires three small, stable changes to MetaGPT's prompt and action flow — no new roles, no per-project code.

Current Engineer Behaviour (Baseline)

In baseline MetaGPT, the Engineer agent receives a system design document, role instructions, and optional repository context. It immediately emits tool calls (e.g., Editor.write) or inline code blocks. There is no mandatory pre-generation verification phase.

Engineer → Editor.write → Files Written

No gate. Errors are discovered post-generation, or silently.

The Three Changes

Change 1: Verification Protocol (Prompt-Level Rule)

A fixed protocol block is prepended to the Engineer (and optionally Architect) prompt:

VERIFICATION PROTOCOL (MANDATORY)

- Before generating any design/code, you MUST produce a CHECK PLAN. - A CHECK PLAN lists every class, method, function, route, file path, table, and queue name you intend to reference.

- For each item, set Origin to one of: EXISTING, NEW, UNKNOWN.

- Rules:

* UNKNOWN: you MUST request a repo lookup (RAG) before proceeding. * EXISTING: you MUST provide evidence (path + signature) from repo lookup before proceeding.

* NEW: you MUST declare where it will be created and you MUST collision-check via repo lookup.

- Do NOT generate any code/design until all required checks are complete and evidence is available.

Change 2: CHECK PLAN (Mandatory Pre-Generation Artifact)

The agent's output contract is modified so the first output must be a CHECK PLAN:

OUTPUT FORMAT (STRICT)

1) CHECK PLAN

2) (After checks complete) Proceed with the main task output

The first output must be a CHECK PLAN.

No tool calls or code are permitted before this.

Change 3: Evidence-Gated Verification Loop (Runtime Enforcement)

When the agent emits a CHECK PLAN and requests verification:

1. The orchestrator executes the required checks (e.g., queries the repository RAG).
2. The results are returned to the agent.
3. Those results are injected back into the agent's prompt as observational evidence.
4. The agent resumes generation using that evidence.
5. If evidence is missing or contradictory, generation is blocked or escalated.

Without this step, the agent can claim it verified something without seeing results; verification becomes advisory; the model may hallucinate compliance. With this step, the agent must operate on concrete evidence, and the orchestrator can block progress until verification completes.

Before and After Execution Flow

Before:

Engineer → Editor.write → Files Written

After:

Engineer → CHECK PLAN → RAG Verification → Evidence Injection →

Engineer → Editor.write → Files Written

One extra controlled loop.

Orchestrator Changes

One additional interception rule:

- If the Engineer output begins with a CHECK PLAN header or CHECK_PLAN JSON → parse it, execute verification, inject results, resume.
- If the Engineer tries to emit Editor.write before CHECK PLAN → reject or retry.

What Does Not Change

- Engineer role semantics.
- No new role is added.

- No project schema changes.
- No changes to PRD or system design formats.
- No changes to downstream agents.

Only the execution gate changes.

End-to-End Example

▶ *Give me an example.*

The following shows the complete CHECK PLAN lifecycle for a Mass Payments upload flow implementation.

Step 1 — Injected Context

The agent has in its prompt:

- **Environment constraints:** Do not reference any class or method unless verified or introduced in this change. Do not redefine existing symbols. Repository RAG is available.
- **Verification protocol:** Every referenced symbol must be classified as EXISTING, NEW, or UNKNOWN, with corresponding verification rules.
- **Project context:** New domain entity planned: PaymentFileUpload. Upload flow uses a background job for CSV processing.

Step 2 — Agent Outputs CHECK PLAN

Before writing any code:

```
CHECK PLAN

  - PaymentFileUpload  -> NEW    -> collision check
  - UploadController   -> UNKNOWN -> RAG lookup
  - ProcessCsvJob      -> UNKNOWN -> RAG lookup
  - CsvValidator       -> UNKNOWN -> RAG lookup
```

No code is generated at this point.

Step 3 — Runtime Executes Verification

The system queries the RAG index and returns:

```
RAG RESULT 1: UploadController
  Found at: app/Http/Controllers/UploadController.php   Public methods: upload(), validate()


RAG RESULT 2: ProcessCsvJob
  Found at: app/Jobs/ProcessCsvJob.php
  Constructor args: ($uploadId)


RAG RESULT 3: CsvValidator   Not found in repository.


RAG RESULT 4: PaymentFileUpload
  No matching class or table found.
```

These results are injected back into the agent prompt.

Step 4 — Agent Updates Classification

Based on evidence:

- PaymentFileUpload → NEW (collision check passed).
- UploadController → EXISTING.
- ProcessCsvJob → EXISTING.
- CsvValidator → NEW (not found in repo).

Step 5 — Agent Proceeds with Generation

The agent can now safely:

- **Reference** UploadController and ProcessCsvJob using their verified paths and methods.
- **Create** new classes PaymentFileUpload and CsvValidator.
- **Avoid** redefining existing symbols or inventing methods that don't exist.

Without the CHECK PLAN, the agent might silently invent UploadController, duplicate

ProcessCsvJob, or assume a validator already exists when it doesn't. With the CHECK PLAN, all assumptions are surfaced, verification happens before generation, and errors are prevented deterministically.

Summary

The entire enforcement mechanism is captured by three concepts and three changes:

Concepts:

- **Constraints** define what must be true (declarative, variable per environment/project).
- **Protocol** defines how compliance is demonstrated (procedural, stable across all projects).
- **Symbol classification** (EXISTING/NEW/UNKNOWN) forces the agent to surface assumptions and verify them.

Changes:

1. **Verification Protocol:** a fixed prompt block establishing that every referenced symbol must be classified, verified, and evidenced before output.
2. **CHECK PLAN:** a mandatory pre-generation artifact listing all symbols with their origin and required verification.
3. **Evidence-gated loop:** the runtime intercepts verification requests, executes them, and injects results back into the agent before generation may proceed.

Agent → CHECK PLAN → Verification Tool (RAG) → Evidence injected → Agent continues → Generation

Constraints define mandatory truth; the CHECK PLAN is the mechanism that forces the LLM to operationalise and prove compliance with that truth before acting.

Appendix – Orchestrator Implementation

Query 1 At 7.2.3 step 5: 'If evidence is missing or contradictory, generation is blocked or escalated.' Which entity performs this check and blocks execution — is it the Engineer agent, the LLM, or something else?

Query 2 At 7.2.3: 'When the agent emits a CHECK_PLAN and requests verification, the orchestrator executes the required checks (e.g., queries the repository RAG).' How is Engineer execution going to be suspended? What is the "orchestrator" and how will it know what action to take? Which part of the MetaGPT framework needs to be extended?

Query 3 "And you add a deterministic dispatcher" — what does this mean concretely? To extend RoleZero? A new class? Does the dispatcher also check the RAG?

Query 4 Referring to the execution flow described in the answers below — where does verification take place? It must happen **AFTER** the RAG is called.

—— **Answers**

---

1. **What is the "Orchestrator"?** → Queries 1 & 2

MetaGPT has no single class literally named "Orchestrator". The term refers to the **MetaGPT runtime execution layer** — the code that sits between the LLM output and the actual file-writes or tool calls. It is **not** the Engineer agent deciding to behave, and **not** the LLM blocking itself. The entity that must *enforce* "block or escalate" is this runtime layer.

In practice it takes two forms, depending on role type:

- **Tool-call / command execution (Architect = RoleZero).** The runtime parses model output into commands and executes them. The execution loop is *RoleZero._act() → parse_commands() → RoleZero._run_commands()*. This is where downstream commands can be refused when evidence is missing or contradictory.
- **SOP action execution (Engineer).** The Engineer does **not** emit *Editor.write* tool JSON; instead it runs *WriteCode*, and the framework saves the result via the repo abstraction (*repo.srcs.save(...)*). The practical block point is therefore **before** WriteCode produces code, or before the framework saves it, e.g., in *Engineer._act_sp_with_cr()* right before *repo.srcs.save(...)*.

The LLM proposes a plan and requests checks; the runtime decides whether the preconditions are satisfied and blocks if not.

2. **What "Suspension" Means in MetaGPT** → Query 2

"Suspension" is **not** a thread pause. It is a **controlled multi-phase execution** where the runtime:

- collects a *CHECK_PLAN*,
- performs verification calls (RAG queries), • injects the results back into the next LLM call,
- only then allows artifact emission.

This is implemented as a **gate in the execution loop**. The LLM doesn't "pause itself"; the framework simply does not call the next step until the gate conditions are met.

3. **The Deterministic Dispatcher** → Queries 2 & 3

A "deterministic dispatcher" is **a small, non-LLM piece of control logic** in the runtime that makes a binary decision about what to do next based on the *shape* of the agent's output — not on any model judgement.

3.1 The gate logic

In essence it is an *if/else* inserted in the execution loop:

- **If** the agent output is a *CHECK_PLAN* (or contains one), **then** run verification first.
- **Else**, execute normal tool calls / proceed with normal action execution.

It is "deterministic" because the decision is made by code (string/JSON parsing + rules), not by the model.

3.2 Dispatcher, Verifier, Injector — separation of responsibilities

The dispatcher **does not** perform the RAG check itself; it **decides to trigger verification** and hands off to a dedicated verifier. Three components work together:

- **Dispatcher (gate)** — detects *CHECK_PLAN*, parses it into structured items, decides "verification required", and routes to the verifier.
- **Verifier (RepoRAG client)** — executes RAG queries and returns evidence objects (hits, file paths, signatures).
- **Injector** — formats evidence into "observations", appends them to the next prompt context, then re-invokes the agent to proceed.

This separation matters because the verifier will be reused in multiple places (both the Architect and Engineer paths).

3.3 Implementation options

Option A — minimal (recommended): extend RoleZero directly. No new class is strictly required. *RoleZero* already sequences: call LLM → parse commands → execute commands. Insert the gate between parsing and execution:

- *commands = parse_commands(llm_output)*
- *if is_check_plan(commands):* → divert to verification
- *else:* → *_run_commands(commands)*

Option B — cleaner codebase: a reusable VerificationGate class. Implement the dispatcher as a small utility component, e.g., *VerificationGate.dispatch(role_output) -> (action, payload)*, and call it from *RoleZero* and later from *Engineer/WriteCode* as well. This avoids duplicated gate logic across role types.

4. Which Parts of MetaGPT to Extend → Query 2

The insertion points differ between the two execution paths.

4.1 Architect path (RoleZero / tool-call)

- Extend **RoleZero._act()** right after *parse_commands()* and before *_run_commands()*.

- If the first command block is a *CHECK_PLAN* request (or your own sentinel), divert into verification; otherwise run normally.

This is the cleanest suspension point because *RoleZero* already parses and executes commands in a single loop.

4.2 Engineer path (SOP / WriteCode)

The Engineer does **not** go through *RoleZero* command execution, so a different insertion point is needed. You cannot intercept *Editor.write* JSON here. Two options:

- **Preferred:** wrap/extend *WriteCode.run()* so it becomes two-stage: (i) request *CHECK_PLAN*, (ii) run RepoRAG verification, (iii) request code with evidence injected.
- **Alternative:** add the gate in *Engineer._act_sp_with_cr()* before persisting output — i.e., do not call *repo.srcs.save* unless verification success is present in context.

In both cases, the same dispatcher+verifier pattern is reused; only the insertion point changes.

5. The Complete Execution Flow → Queries 3 & 4

The following describes the full cycle for the tool-call / *RoleZero* path. The Engineer path follows the same logic but at a different insertion point (see Section 4.2).

5.1 Step-by-step: RoleZero (Architect / tool-call)

- LLM output arrives.
- *parse_commands()* produces a command list.
- **Dispatcher checks**: does the output contain a *CHECK_PLAN*?
- **If yes — enter verification phase:**

  - **Suspend** — do **not** call *_run_commands*.
  - Call *RepoVerifier.verify(check_plan_items)* — this runs RAG then evaluates (see 5.2 below).
  - Store result as *verification_evidence*.
  - Re-prompt the same role with evidence injected.

- **If no** — no *CHECK_PLAN* detected → call *_run_commands* as normal.

That is the "suspend/resume" mechanism. It is **not** a thread pause; it is simply a refusal to call the next step until the gate conditions are satisfied.

5.2 Where verification happens — the two-phase call

Verification happens **inside the runtime, immediately after the RAG call returns**. It is never performed by the LLM. *RepoVerifier.verify(check_plan_items)* should be understood as two sequential sub-steps:

5.2a Evidence acquisition (RAG call)

- The runtime calls *RepoRAG.lookup_symbol(...)*,

RepoRAG.signature_lookup(...), *RepoRAG.collision_check(...)* based on the *CHECK_PLAN* items.

- Output: raw evidence — hits, file paths, signatures, commit refs, etc.

5.2b Verification (evaluate evidence against CHECK_PLAN + constraint bindings) This step executes **after** the RAG calls return. It applies deterministic rules:

- If *origin* == *EXISTING* → require ≥1 matching hit and (if requested) matching signature.
- If *origin* == *NEW* → require collision-check returns "no match".
- If evidence is ambiguous or contradictory → mark failure and block.

The result is a *VerificationReport { pass/fail, evidence, issues }*. Then:

- **Pass:** inject the evidence + report into the next prompt and re-prompt the role.
- **Fail:** block execution and re-prompt with the failures / required corrections.

In short: RAG returns raw evidence; verification is the **deterministic code-level evaluation** of that evidence against the *CHECK_PLAN* and policies. The LLM is not involved in this decision.

6. Summary

- **The "orchestrator"** is the MetaGPT execution layer (*RoleZero* for tool-call roles; Engineer/Action runner + repo save path for SOP roles), not the LLM.
- **Blocking** is enforced by that runtime layer, not by the Engineer agent or LLM deciding to self-constrain.
- **Suspension** is a multi-phase gate: generation does not proceed until

CHECK_PLAN is collected, RAG evidence is retrieved, verification passes, and evidence is injected.

- **Dispatcher** = the *if/else* gate logic, inserted into *RoleZero._act()* (for Architect) and/or *WriteCode.run()* / *Engineer._act_sp_with_cr()* (for Engineer). Either extend *RoleZero* directly or create a reusable *VerificationGate* class.
- **Verifier** = the separate component that actually performs the RAG call and then evaluates the results. The dispatcher routes to the verifier; it does not perform the check itself.
- **Verification** happens **after** the RAG call returns — always in runtime code, never by the LLM.